

Exceptions & Asertions

Programación JAVA.

Objetivos

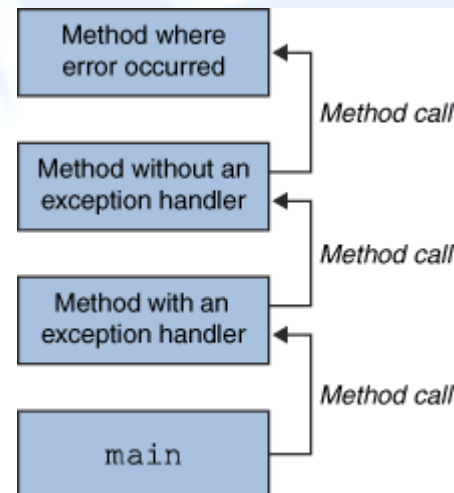
- Definir excepciones
- Usar el bloque try, catch y finally
- Describir las distintas categorías de excepciones
- Identificar las excepciones más comunes
- Desarrollo de programas que manejan excepciones definidas por el usuario
- Características y uso de aserciones
- Uso de las aserciones: apropiados e inapropiados
- Activar aserciones en tiempo de ejecución

Excepciones y Aserciones

- Las excepciones son los mecanismos usados por algunos lenguajes de programación para definir qué hacer cuando algo inesperado ocurre: error en un algoritmo de búsqueda, invocación de un método con argumentos no aceptables, fallo en la conexión de red, etc
- Un *excepción* es un evento, que se produce durante la ejecución de un programa, que interrumpe el flujo normal de las instrucciones del programa
- Una aserción es un tipo de declaración en Java que permite poner a prueba las suposiciones sobre la lógica de un programa. Comprobar que en un punto en particular del código que una variable será siempre positiva.
- Las suposiciones son expresiones lógicas que se evalúan cuando la aserción se ejecuta. Si la suposición lógica no es cierta se desencadena un error
- Las aserciones se emplean generalmente en las fases de implementación y pruebas de software y pueden ser desactivadas cuando el producto está en fase de producción

Excepciones

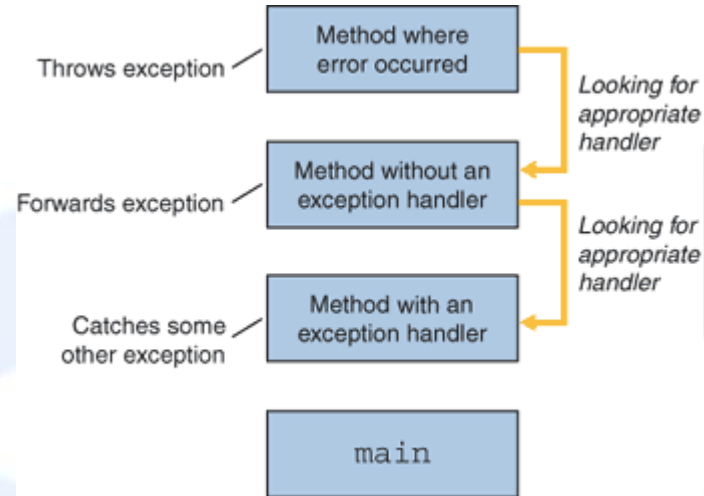
- Cuando se produce un error dentro de un método, el método crea un objeto y lo entrega al sistema de tiempo de ejecución. El objeto, llamado *objeto de excepción*, contiene información sobre él
- Si en un método se produce un error, se lanza (throws) una excepción en dónde el sistema en tiempo de ejecución trata de encontrar un manejador., el sistema de ejecución de los intentos de encontrar algo para manejarlo
- Se crea un conjunto por orden de los métodos que han sido llamados hasta el método que lanzó la excepción con el objetivo de poder controlarla. La lista de los métodos llamados se conoce como la *pila de llamadas* (call stack)



Excepciones

- El entorno de ejecución accede a la pila de llamadas para un método que contiene un bloque de código que puede manejar la excepción. Este bloque de código se denomina *manejador de excepciones*.
- La búsqueda se inicia en el método en que se produjo el error y a través de la pila de llamadas. Si se encuentra un controlador adecuado, el sistema de ejecución pasa a la excepción al controlador. Un controlador de excepciones se considera adecuado si el tipo de la excepción del objeto lanzado coincide con el tipo que puede ser manejado por el controlador
- El controlador de excepciones escogido se dice que *captura (catch) la excepción*. Si el entorno de ejecución no es capaz de encontrar una manejador a través de los métodos de la pila el programa termina.

Excepciones



Tipos de Excepciones

- En Java hay dos grandes categorías de excepciones:
 - **Checked exceptions:** son condiciones excepcionales que se esperan manejar dentro del programa en donde una aplicación bien escrita se debe anticipar y recuperar.
Ejemplo: aplicación que lee un archivo y este no se encuentra
 - **Unchecked exceptions:**
 - Condición excepcional provocada por bug (error de programación) en el código. Manejo difícil dentro de un programa. Se denominan unchecked debido a que no hay que no hay que manejarlas de una forma especial. Las situaciones de excepción provocadas por bug se llaman excepciones en tiempo de ejecución (runtime exceptions) → acceso a un índice de un array no válido.
 - Excepciones que son provocadas por un problema ajeno al propio programa en ejecución → error en el hw del equipo accediendo a la red
- La clase Exception es la clase base de las excepciones Checked y Unchecked
- La clase Error es la clase base usada dentro de las Unchecked exceptions para indicar condiciones de error graves. No se espera que el sistema se recupere de este tipo de excepciones → El programa termina → se intenta recuperar lo máximo el trabajo del usuario

Tipos de Excepciones.

- La clase RuntimeException es la clase base usada dentro de las Unchecked exceptions para indicar condiciones de excepción que con el resultado de errores en el código (bug) → se termina el programa y se intenta recuperar el trabajo del usuario.
- Excepciones de tipo Check → examen el API

Captura y gestión de excepciones

- Java proporciona un mecanismo para el manejo de excepciones → try .. catch

```
try {  
    //Código en el que se pueden producir excepciones  
}catch( ExceptionType exception){  
    //Código que se ejecuta si ExceptionType es lanzada  
}
```

- El código dentro de try puede lanzar excepciones (throw) → en cuanto lanza una excepción termina
- Dentro del bloque catch se maneja la excepción → Tipo de la excepción lanzada (throw) = tipo de excepción del bloque catch

Captura y gestión de excepciones

```
public class AddArguments{  
    public static void main(String[] args){  
        int suma=0;  
        for(int i=0;i<args.length;i++){  
            suma += Integer.parseInt(args[i]);  
        }  
        System.out.println("La suma es: " + suma);  
    }  
}  
java AddArguments 1 2 3 cuatro 5
```

Resultado de la ejecución:

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "cuatro"  
    at java.lang.NumberFormatException.forInputString (NumberFormatException.java:48)  
    at java.lang.Integer.parseInt (Integer.java:447)  
    at java.lang.Integer.parseInt (Integer.java:497)  
    at AddArguments.main (AddArguments.java:5)
```

Captura y gestión de excepciones

```
Public class AddArguments{
    Public static void main(String[] args){
        int suma=0;
        try{
            for(int i=0;i<args.length;i++){
                suma += Integer.parseInt(args[i]);
            }
            System.out.println("La suma es: " + suma);
        }catch(NumberFormatException e){
            System.out.println("Uno de los argumentos de " +
                "la línea de comandos no es un número");
        }
    }
}

java AddArguments 1 2 3 cuatro 5
```

Captura y gestión de excepciones

- Dentro de un bloque try .. catch puede haber múltiples cláusulas catch

```
try{  
    //Código puede lanzar excepciones  
}catch(Exception1 e){  
    //Codigo que maneja Exception1  
}catch(Exception2 e){  
    //Codigo que maneja Exception2  
}catch(Exception e){  
    //Codigo que maneja cualquier excepción lanzada  
}
```

- El orden de las sentencias catch importa.
- Se ejecuta el primer catch que es capaz de manejar la excepción lanzada → el tipo de la excepción lanzada debe ser igual al que maneja la cláusula catch o un subtipo

Captura y gestión de excepciones

- Ejemplos de uso del bloque try .. Catch

```
// Bloque 1
try {
// Bloque 2
} catch (Exception error) {
// Bloque 3
}
// Bloque 4
-----
// Bloque 1
try {
// Bloque 2
} catch (ArithmeticException ae) {
// Bloque 3
} catch (NullPointerException ne) {
// Bloque 4
}
// Bloque 5
```

Sin excepciones: 1 → 2 → 4

Excepción en el bloque 2: 1 → 2* → 3 → 4

Con una excepción en el bloque 1: 1

Sin excepciones: 1 → 2 → 5

Excepción de tipo aritmético: 1 → 2 → 3 → 5

Acceso a un objeto nulo (null): 1 → 2 → 4 → 5

Excepción de otro tipo diferente: 1 → 2

Captura y gestión de excepciones

- Ejemplos de uso del bloque try .. Catch

```
// Bloque 1
try {
// Bloque 2
} catch (ArithmeticException ae) {
// Bloque 3
} catch (Exception error) {
// Bloque 4
}
// Bloque 5
```

Sin excepciones: $1 \rightarrow 2 \rightarrow 5$

Excepción de tipo aritmético: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$

Excepción de otro tipo diferente: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$

Mecanismo de la gestión de la pila

- Una sentencia de método lanza una excepción → si no se maneja en el propio método, se lanza (throws) al método que lo ha llamado.
- El proceso continúa hasta:
 - Hay un método que maneja la excepción
 - Se llega al método main:
 - Se maneja la excepción → finalización normal
 - No se maneja la excepción → Finalización anormal del programa

Mecanismo de la gestión de la pila

```
public class CallStack {
    public void method1(){
        try{
            method2();
        }catch (NumberFormatException e){
            System.out.println("Excepción capturada
en Metodo1");
        }
    }
    public void method2(){
        //Some code that can throws a exception
        throw new
        java.lang.NumberFormatException("Excepcion en
metodo2");
    }
}
```

```
public static void main(String[] args){
    System.out.println("Comienzo de la
ejecución del programa");
    CallStack stack = new
    CallStack();
    stack.method1();
    System.out.println("Fin del
programa");
}
```

Mecanismo de la gestión de la pila

- Estado de la pila:

method2() → Excepción
method1()
main()

method1() → manejo de la excepción
main()

Mecanismo de la gestión de la pila

```
1. public class CallStack {  
2.     public void method1(){  
3.         method2();  
4.     }  
5.     public void method2(){  
6.         //Some code that can throws a  
           exception  
7.         throw new  
           java.lang.NumberFormatException(  
           "Excepcion en metodo2");  
8.     }
```

```
9.     public static void main(String[] args){  
10.        System.out.println("Comienzo de la  
           ejecución del programa");  
11.        CallStack stack = new CallStack();  
12.        stack.method1();  
13.        System.out.println("Fin del programa");  
14.    }  
15. }
```

Ejecución:

Comienzo de la ejecución del programa

Exception in thread "main" java.lang.NumberFormatException:
Excepcion en metodo2

at CallStack.method2(CallStack.java:7)

at CallStack.method1(CallStack.java:3)

at CallStack.main(CallStack.java:12)

Mecanismo de la gestión de la pila

- Estado de la pila:

method2() → Lanzamiento de NumberFormatException
method1()
main()

method1(): No hay manejador
main()

main(): No hay manejador → Terminación del programa
--

Clausula Finally

- El bloque finally *siempre* se ejecuta, incluso si se produce una excepción inesperada.
- Algo más que el manejo de excepciones → permite al programador ejecutar código independiente si se produce una excepción. Ejemplo: código de liberación de recursos como el cierre de la conexión a la base de datos, cierre de un descriptor de fichero, etc

```
//código
Try{
    //Codigo que puede lanzar una excepción
}catch(Exception1 e){
    //manejo de la Exception1
} catch(Exception2 e){
    //manejo de la Exception2

...
}catch(Exception e){
    //manejo del resto de excepciones
}finally{
    //Codigo de liberación de recursos
}
```

Clausula Finally

- Sólo hay una situación en donde no se ejecuta la cláusula finally → invocación de salida del programa mediante `System.exit(0)`

```
public void writeList() {  
    PrintWriter out = null;  
    try {  
        System.out.println("Entering try  
statement");  
        out = new PrintWriter( new  
            FileWriter("OutFile.txt"));  
        for (int i = 0; i < SIZE; i++)  
            out.println("Value at: " + i +  
                " = " + vector.elementAt(i));  
    } catch (ArrayIndexOutOfBoundsException e) {  
        System.err.println("Caught " +  
            "ArrayIndexOutOfBoundsException: "  
            " + e.getMessage());  
    } catch (IOException e) {  
        System.err.println("Caught  
IOException: " +  
            e.getMessage());  
    } finally {  
        if (out != null) {  
            System.out.println("Closing  
PrintWriter"); out.close();  
        } else {  
            System.out.println("PrintWriter  
not open");  
        }  
    }  
}
```

Clausula try-with-resources

- Antes de la versión 7 de JSE, la gestión de excepciones para la gestión de recursos (ficheros, conexiones a bases de datos,...) presentaba un problema añadido en la liberación de recursos.

- La estructura utilizada era la siguiente:

```
InputStreamReader lector = null;
try {
    lector = new InputStreamReader(System.in);
    int dato = lector.read();
} catch (IOException e) {
    System.out.println("Excepción al leer");
} finally {
    try {
        if (lector != null) {
            lector.close();
        }
    } catch (IOException e) {
        System.out.println("Excepción al cerrar");
    }
}
```

Clausula try-with-resources

- Para simplificar esta estructura, se introduce la clausula try-with-resources en la versión 7

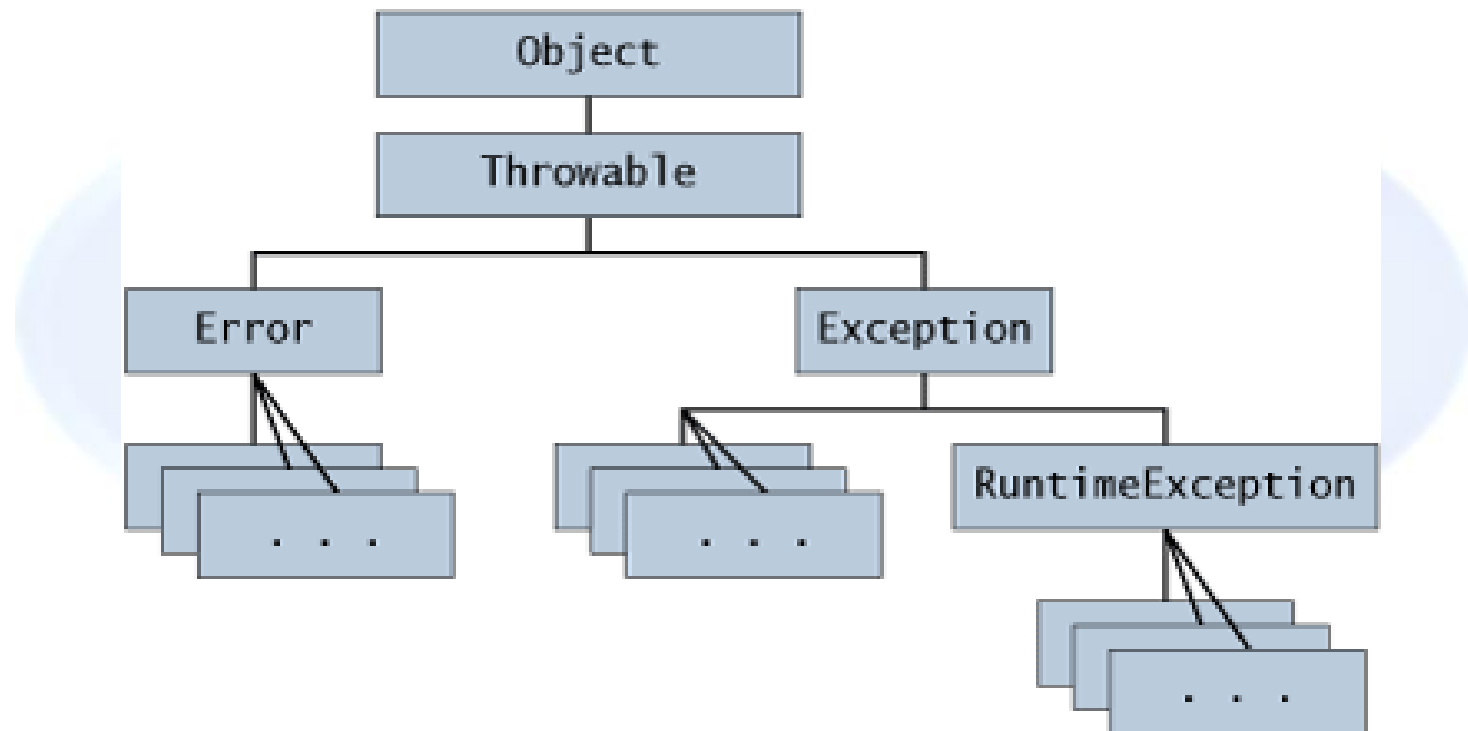
```
try (InputStreamReader lector = new InputStreamReader(System.in)) {  
    int dato = lector.read();  
} catch (IOException e) {  
    System.out.println("Excepción al leer");  
}
```

- En esta forma de uso, el recurso que se libera se declara en la clausula try, encargándose automáticamente la JVM de cerrar el recurso cuando se finalice.
- Se pueden introducir varios recursos dentro de la clausula try, separados por punto y coma.

Categorías de las excepciones

- **Throwable:** Clase base que representa todo lo que se puede “lanzar” en Java y capturar empleando los mecanismos try .. catch
 - Contiene una instantánea del estado de la pila en el momento en el que se creó el objeto (“stack trace” o “call chain”)
 - Almacena un mensaje (variable de instancia de tipo String)
 - Puede tener una causa, también de tipo Throwable, que permite representar el error que causó este error
- Subclases: las principales subclases son Error, RuntimeException y Exception
- No se debe emplear Throwable → uso de las subclases
- **Error:** Subclase de Throwable que indica problemas graves que una aplicación en dónde recuperarse del problema es bastante difícil. *Ejemplos:* Memoria agotada, error interno de la JVM...
- RuntimeException → problema de diseño o implementación → es preferible una terminación del programa que capturar excepción
- Otras excepciones → deben de capturarse, pueden ocurrir con frecuencia. Ejemplo: Fichero no encontrado, url no valida, etc

Categorías de las excepciones



Excepciones comunes

- En java hay excepciones predefinidas en el sistema (ver API). Algunas de las más empleadas son:
 - `NullPointerException` → Excepción que se lanza cuando se intenta acceder a un atributo o método usando una variable que no referencia a un objeto. Ejemplo:

```
Employee e = null;  
e.getSalary() → throws NullPointerException
```
 - `FileNotFoundException` → Lectura de un fichero que no existe
 - `NumberFormatException` → Intento de convertir un `String` en un número que tiene un formato invalido
 - `ArithmeticException`: División por cero
 - `SecurityException` → Acceso a un sistema de ficheros sin permisos
 - `SQLException` → Excepciones lanzadas como resultado de operar con una base de datos. Ejemplo: ejecutar una consulta con una conexión cerrada a la base de datos

Tratamiento de excepciones

- Código robusto → tratamiento de las Checked exceptions que se puedan producir en cualquier parte del código
- Tipos de acciones que se pueden tomar:
 - Manejo de la excepción con un bloque try .. catch .. Finally
 - Declarar excepciones que el método donde se produce la excepción es capaz de lanzar (Throws).
 - Cuerpo de un método se lanza una excepción (subclase de Exception), en la cabecera del método hay que añadir una cláusula throws que incluye una lista de los tipos de excepciones que se pueden lanzar dentro del metodo
 - *Ejemplo*
 - `public String leerFichero (String nombreFicherothrows IOException`
- Las unchecked exception no es necesario declararlas (Error y RuntimeException) → hay excepciones a esta regla → uso de NumberFormatException

Sobreescritura de métodos

- Sobreescritura de método que pueden lanzar excepciones → El metodo sobreescrito debe lanzar el mismo tipo de excepción o un subtipo, nunca una superclase

```
public class TestA {  
    public void methodA() throws RuntimeException {  
        // do some number crunching  
    }  
}  
  
public class TestB1 extends TestA {  
    public void methodA() throws ArithmeticException {  
        // do some number crunching  
    }  
}  
  
public class TestB2 extends TestA {  
    public void methodA() throws Exception {  
        // do some number crunching  
    }  
}
```

Creación de Excepciones

- Crear excepciones propias → Extender de la clase Exception

```
public DivideByZeroException extends ArithmeticException{
    public DivideByZeroException(String Message){
        super(message);
    }
}

Public class ServerTimeOutException extends Exception{
    private int port;
    ServerTimeOutException(String msg,int port){
        super(msg);
        this.port = port;
    }
    public int getPort(){
        return this.port;
    }
}
```

Manejo de Excepciones creadas por el usuario

- Una excepción creada por el usuario se maneja igual que las excepciones del sistema:
 - Manejo de la excepción en bloques try .. catch .. Finally
 - Puede ser lanzada en el cuerpo de un método

```
public void connect(String server) throws ServerTimeoutException{
    boolean successful;
    int port=80;
    successful = open(server,port);
    if (!successful){
        throw new ServerTimeoutConnection("Could not connect",port);
    }
}

public void findServer() {
    try{
        conect(DEFAULTSERVER);
    }catch(ServerTimeoutException e){
        System.out.println("Error: " + e.getMessage() +
            " connecting to port " + e.getPort());
    }
}
```

Manejo de Excepciones creadas por el usuario

```
public void findServer() {  
    try{  
        conect(DEFAULTSERVER);  
    } catch (ServerTimeoutException e){  
        System.out.println("Server timeout, trying alternative);  
        try{  
            connect(ALTERATIVESERVER);  
        } catch (ServerTimeoutException e){  
            System.out.println("Error: " + e.getMessage() +  
                " connecting to port " + e.getPort());  
        }  
    }  
}
```

Assertions

- Formas sintácticas permitidas.

```
assert Expression1
```

```
assert Expression1 : Expression2
```

- *Expresión1* es una expresión booleana.
- *Expression2* es cualquier tipo que se convierte a String y proporciona un mensaje
- Se evalúa la expresión y si es falsa genera una excepción `AserciónError`
- Objetivo: Detalles de error en fase de implementación. No debe de intentar solventar errores
- Tratamiento de la excepción `AserciónError` como una excepción más
- Posibilidad de desactivarse para entornos de producción
- Ejecución en JDK 1.3 debe de compilarse con la opción `-source 1.3`

Usos recomendados

- Método para documentar el código → se establecen las suposiciones del programador
- Especialmente útil en la fase de mantenimiento / desarrollo con varios grupos de trabajo activos
- Verificar la lógica interna de un método o un conjunto de métodos fuertemente acoplados. No deben de verificar que el código se usa de forma correcta
- Ejemplos de usos recomendados de Assertions son.
 - Invariantes internas
 - Invariantes de control de flujo
 - Postcondiciones e invariantes de clase

Usos no recomendados

- Comprobar los parametros de un método → empleo de excepciones, capturando la causa de excepción y lanzando la excepción apropiada (IllegalArgumentException, NullPointerException)
- Estado de las Assertions en un programa en producción: desactivadas → si se realiza trabajo de control de parametros de un metodo y están inactivas → no hay control

Control de Assertions

- Se pueden desactivar → Excepciones no
- No hay sobrecarga motivada por la comprobación de las Assertion en producción → desactivadas
- Por defecto: Assertions desactivadas
- Habilitar Assertions: `-enableassertions` o `-ea`
`Java -enableassertions MiPrograma`
- *Opciones:*
 - *sin argumentos* → Activa o desactiva las afirmaciones en todas las clases, excepto las clases del sistema.
 - *packageName* → Activa o desactiva las afirmaciones en el paquete llamado y cualquier sub-paquetes.
 - *className* → Activa o desactiva las afirmaciones de la clase llamada